

Greenfoot frontend for TicTacToe REST server

CS 141

Spoiler alert:

- Not that much code to write
- But it all connects up in sophisticated ways

Goals

- Remote Procedure Call (RPC) over HTTP (“the web”)
 - often called REST (representational state transfer)
- JSON for client/server data exchange
- Connector: Java class abstracts away complexity of RPC
- GUI (graphical user interface) with Greenfoot to play TicTacToe against a server

TicTacToe REST API: 3 “functions”

Assume server is `http://euclid.knox.edu:8082`

`/tictactoe/newgame?user=username`

`/tictactoe/move?user=username&row=0&col=0`

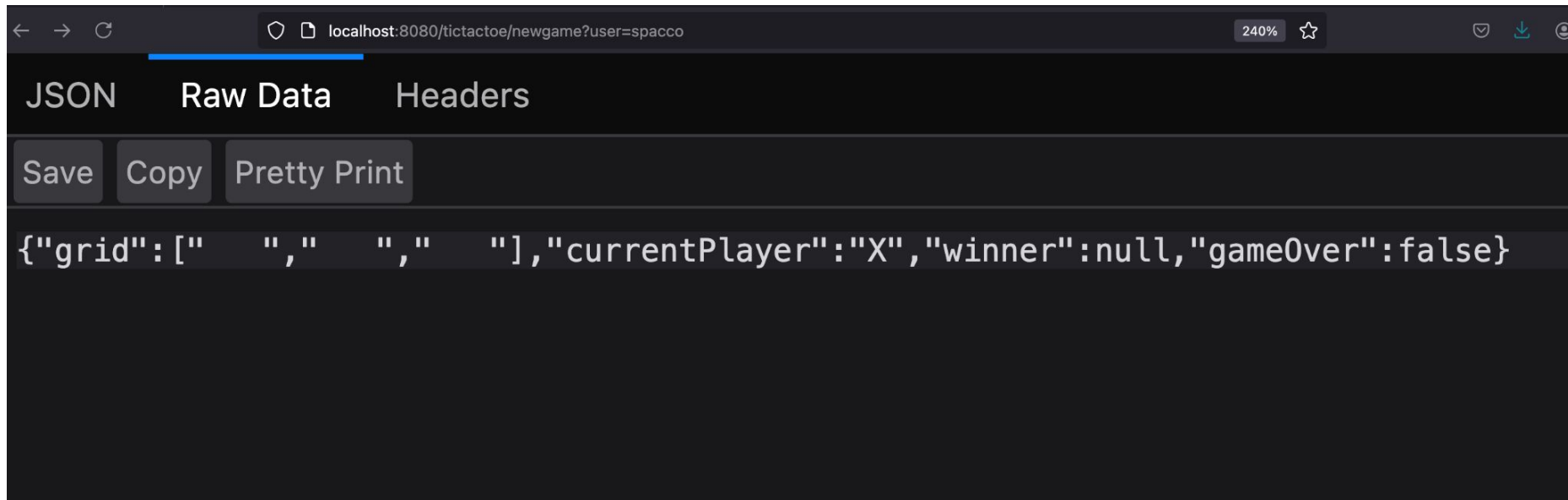
`/tictactoe/state?user=username`

Server responds with JSON

Let's interact with the server

/tictactoe/newgame?user=spacco

Showing raw JSON response



JSON (JavaScript Object Notation)

- Basically, a structured text format that programming languages use to exchange data
- Based on JavaScript syntax, but can be read by any programming language

JSON example

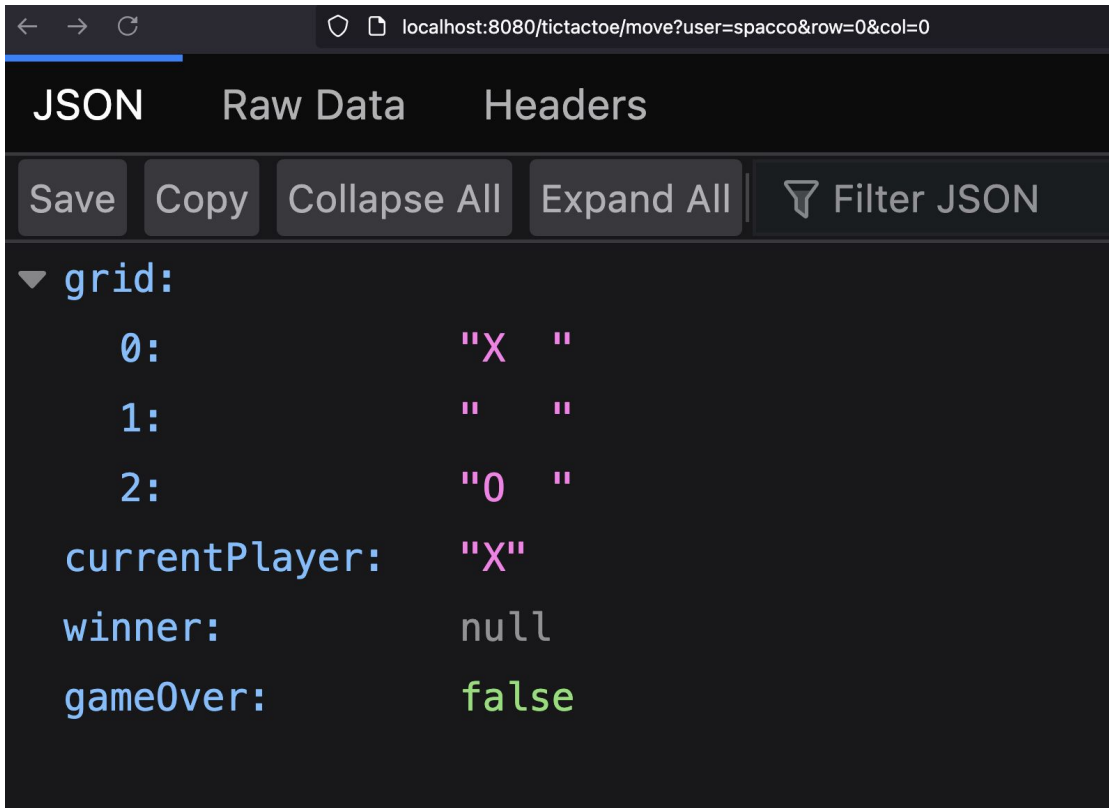
```
{  
  "grid": [  
    "  ",  
    "  ",  
    "  ",  
  ],  
  "currentPlayer": "X",  
  "winner": null,  
  "gameOver": false  
}
```


/tictactoe/move?user=spacco&row=0&col=0

Put X at 0,0 (top left)

server random move for O

using JSON pretty
printer in Firefox



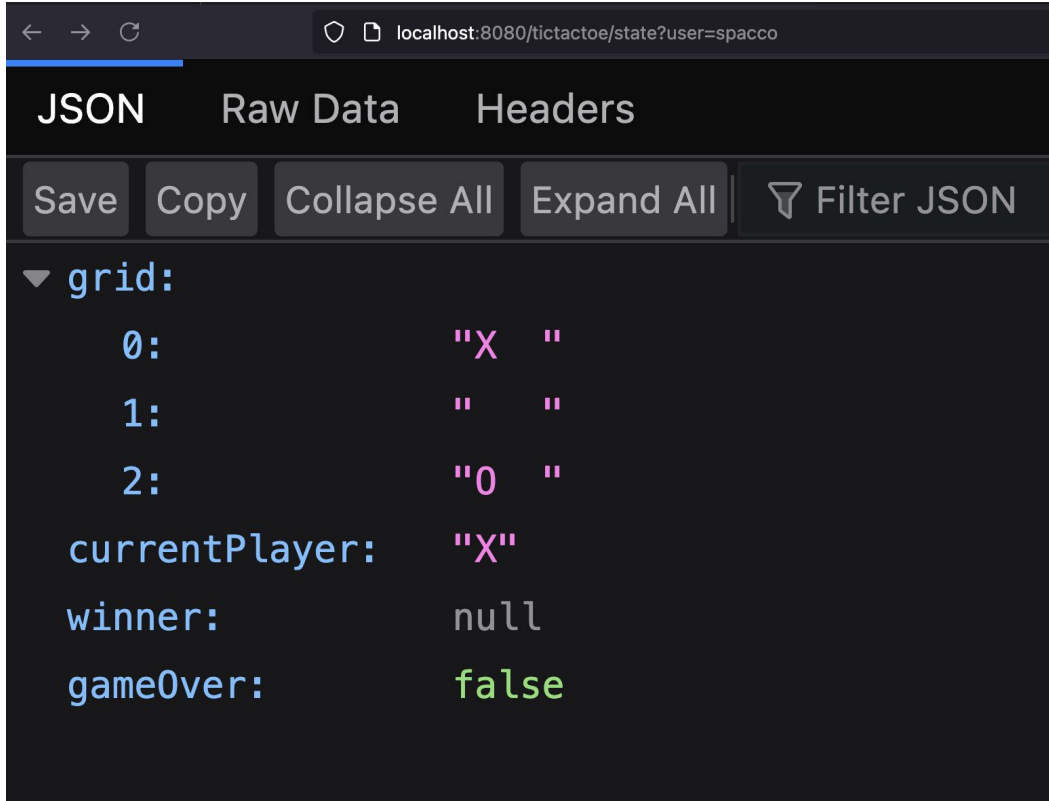
The screenshot shows a web browser window with the address bar displaying `localhost:8080/tictactoe/move?user=spacco&row=0&col=0`. The browser's developer tools are open, showing the JSON response in the 'JSON' tab. The response is a JSON object with the following structure:

```
{  "grid": {    "0": "X",    "1": "",    "2": "O"  },  "currentPlayer": "X",  "winner": null,  "gameOver": false}
```

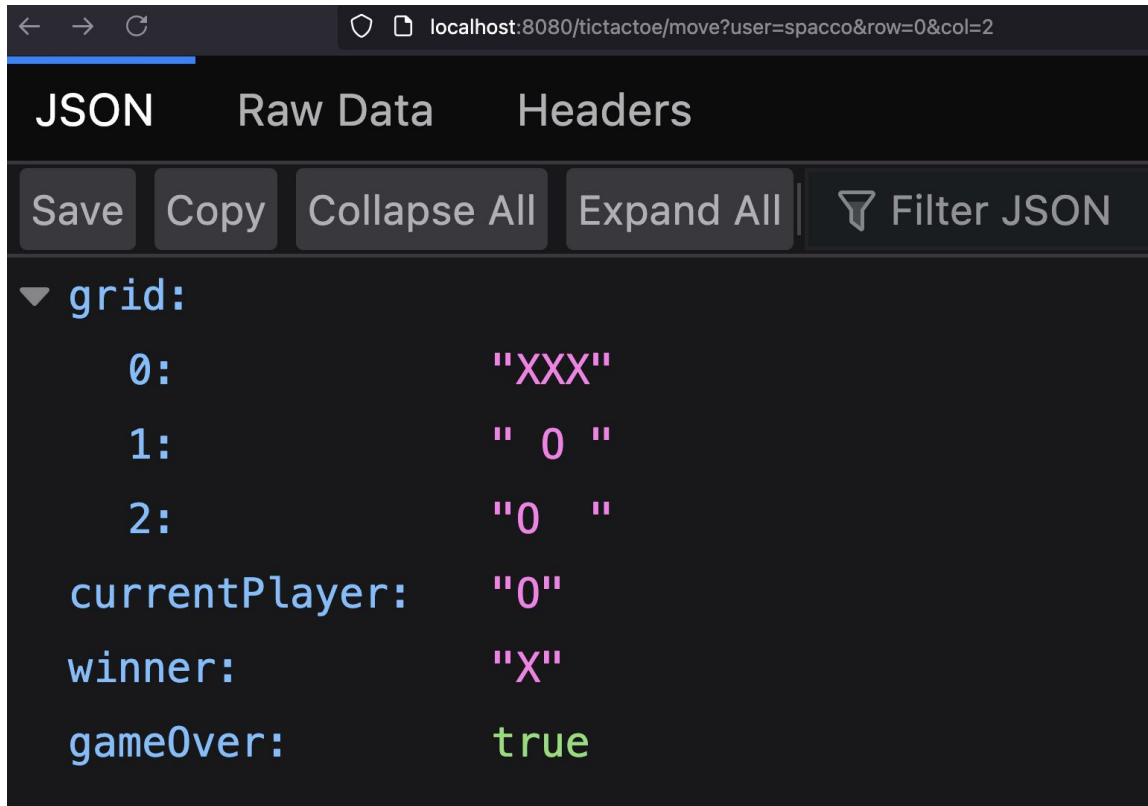
The JSON is formatted with syntax highlighting: strings are in pink, numbers in blue, and booleans in green. The 'grid' object is expanded, showing a 3x3 grid state. The 'currentPlayer' is 'X', 'winner' is null, and 'gameOver' is false.

/tictactoe/state?user=spacco

Check current game state for spacco



/tictactoe/move?user=spacco&row=0&col=2
X (player) wins because O (server) moves randomly



```
localhost:8080/tictactoe/move?user=spacco&row=0&col=2

JSON Raw Data Headers

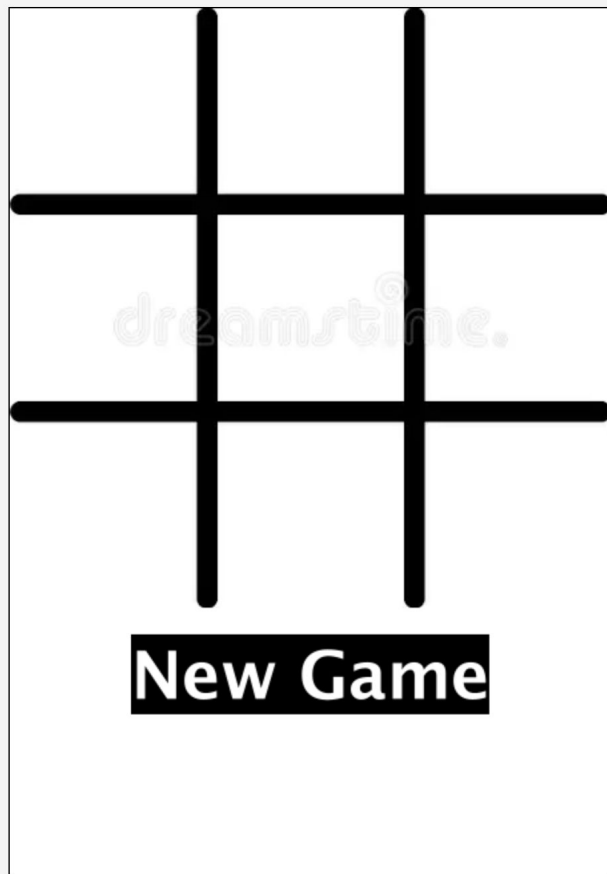
Save Copy Collapse All Expand All Filter JSON

▼ grid:
  0:      "XXX"
  1:      " 0 "
  2:      "0  "
currentPlayer: "0"
winner:      "X"
gameOver:    true
```

Playing in the web browser is not very fun

Let's build a GUI with Greenfoot

Greenfoot project for students



> Act

|| Pause

↺ Reset

Speed:

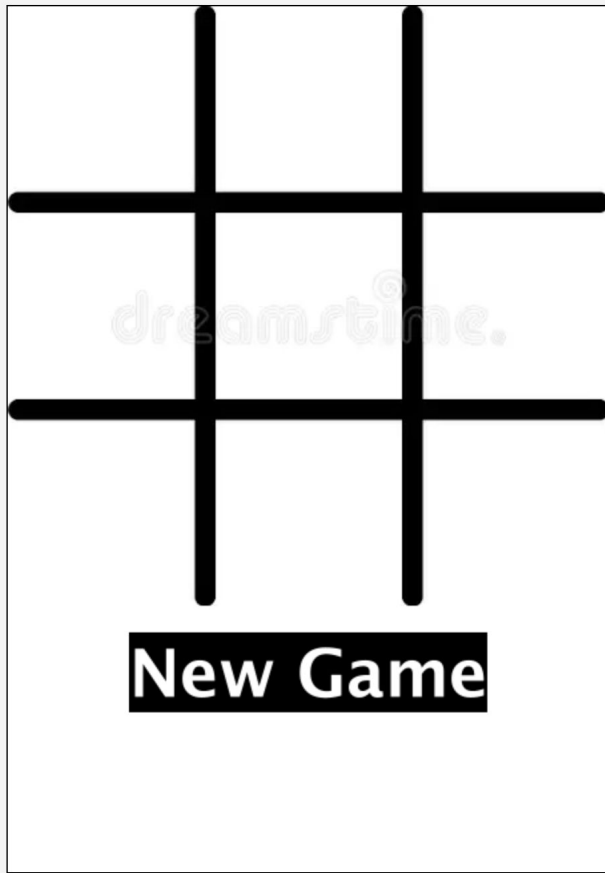
Share...

World
MyWorld

Actor
Button
Letter

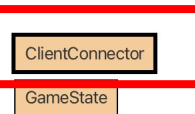
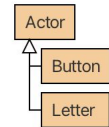
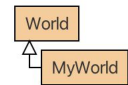
ClientConnector

GameState



ClientConnector
abstracts the URLs
into Java functions

Share...



> Act

|| Pause

↺ Reset

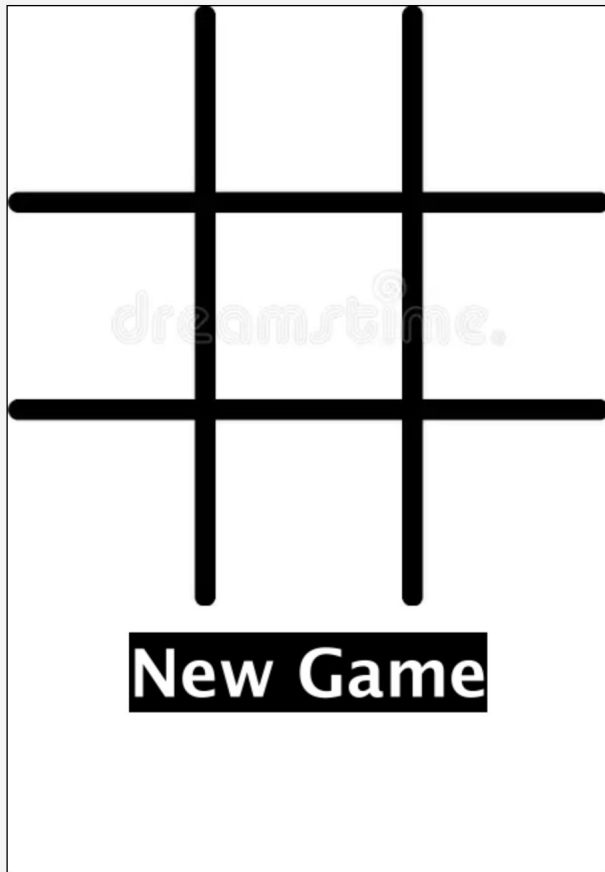
Speed:

This is written for you — we wouldn't ask you to write this but it's understandable at a high-level

```
// start a new game
public GameState newGame()
{
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(baseUrl + "/newgame" + "?user=" + user))
        .GET()
        .build();
    try {
        HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
        return GameState.fromJson(response.body());
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

Also given to students, also understandable even if you couldn't easily write this

```
public GameState makeMove(int row, int col)
{
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(baseUrl + "/move" + params + "&row=" + row + "&col=" + col))
        .GET()
        .build();
    //System.out.println("Request: " + request);
    try {
        HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
        return GameState.fromJson(response.body());
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

> Act

|| Pause

↺ Reset

Speed:

Share...

World
MyWorld

Actor
Button
Letter

GameState
matches the JSON
response from the
server

ClientConnector

GameState

GameState also given to students

deserializing JSON is *easy but not simple*

```
public class GameState {
    private char[][] grid = new char[3][3];
    private char currentPlayer = 'X';
    private String winner = null;
    private boolean gameOver = false;

    public static GameState fromJson(String json) {
        Gson gson = new GsonBuilder()
            .registerTypeAdapter(char[][].class, new CharGridDeserializer())
            .create();
        return gson.fromJson(json, GameState.class);
    }

    private static class CharGridDeserializer implements JsonDeserializer<char[][]> {
        @Override
        public char[][] deserialize(JsonElement json, Type typeOfT, JsonDeserializationContext context
            throws JsonParseException
        {
            JsonArray jsonArray = json.getAsJsonArray();
            char[][] result = new char[jsonArray.size()][3];
            for (int i = 0; i < jsonArray.size(); i++) {
                String row = jsonArray.get(i).getString();
                result[i] = row.toCharArray();
            }
            return result;
        }
    }
}
```

GameState also given to students

deserializing JSON is *easy but not simple*

```
public class GameState {  
    private char[][] grid = new char[3][3];  
    private char currentPlayer = 'X';  
    private String winner = null;  
    private boolean gameOver = false;  
  
    public static GameState fromJson(String json) {  
        Gson gson = new GsonBuilder()  
            .registerTypeAdapter(char[][].class, new CharGridDeserializer())  
            .create();  
        return gson.fromJson(json, GameState.class);  
    }  
  
    private static class CharGridDeserializer implements JsonDeserializer<char[][]> {  
        @Override  
        public char[][] deserialize(JsonElement json, Type typeOfT, JsonDeserializationContext context)  
            throws JsonParseException  
        {  
            JsonArray jsonArray = json.getAsJsonArray();  
            char[][] result = new char[jsonArray.size()][3];  
            for (int i = 0; i < jsonArray.size(); i++) {  
                String row = jsonArray.get(i).getString();  
                result[i] = row.toCharArray();  
            }  
            return result;  
        }  
    }  
}
```

```
public class MyWorld extends World
```

```
{
```

```
    ClientConnector connector;
```

```
    String user = "test";
```

```
    String url = "http://localhost:8080/tictactoe";
```

```
    Button newGame = new Button("New Game");
```

```
    Button gameOver;
```

```
public MyWorld() throws Exception
```

```
{
```

```
    // Create a new world with 450x450 cells with a cell size of 1x1 pixels.
```

```
    super(450, 650, 1);
```

```
    // set background image
```

```
    // we picked 450x450 because the image is 450x450
```

```
    GreenfootImage image = new GreenfootImage("images/grid.png");
```

```
    getBackground().drawImage(image, 0, 0);
```

```
    // the connector for connecting to the game
```

```
    connector = new ClientConnector(url, user);
```

```
    // new game button
```

```
    addObject(newGame, 225, 500);
```

```
    // start a new game
```

```
    newGame();
```

```
}
```

Place tic tac
toe grid image
at top

create client
connector class
to talk to server

add new game
button

start a new
game

Useful methods in MyWorld

```
public void newGame()  
{  
    connector.newGame();  
    removeObjects(getObjects(Letter.class));  
    removeObject(gameOver);  
}
```

```
public void setLetter(int row, int col, char letter)  
{  
    Letter let = new Letter(letter);  
    addObject(let, col * 150 + let.getWidth()/2, row * 150 + let.getHeight()/2);  
}
```

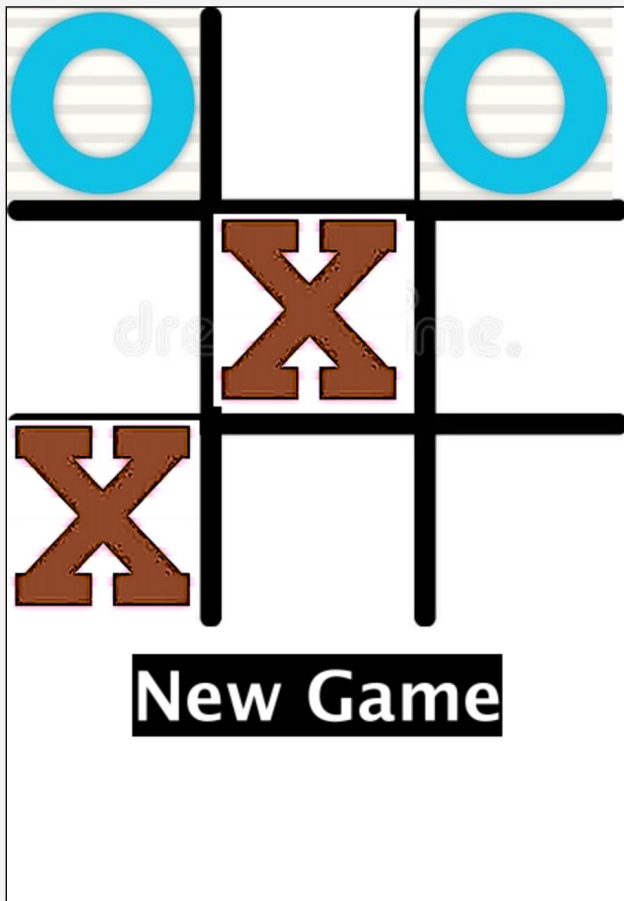
```

public void act(){
    MouseInfo mouse = Greenfoot.getMouseInfo();
    if (mouse != null) {
        if (Greenfoot.mousePressed(null)) {
            int button = mouse.getButton();
            int row = mouse.getY() / 150;
            int col = mouse.getX() / 150;

            if (row < 0 || row > 2 || col < 0 || col > 2) return;
            try {
                GameState state = connector.makeMove(row, col);
                char[][] grid = state.getGrid();
                for (int r=0; r<grid.length; r++) {
                    for (int c=0; c<grid[r].length; c++) {
                        if (grid[r][c] != ' ') setLetter(r, c, grid[r][c]);
                    }
                }
                if (state.isGameOver()) {
                    String winner = state.getWinner();
                    if (!winner.equals("Tie")) winner = "Winner is " + winner;
                    gameOver = new Button(winner);
                    addObject(gameOver, 225, 600);
                }
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}

```

act() method
in MyWorld
figures out
where we
clicked, tells
server the
move,
handles
server's
response



Share...

World
MyWorld

Actor
Button
Letter

Button and Letter
are what they sound
like

ClientConnector
GameState

> Act

|| Pause

🔄 Reset

Speed:



TicTacToe is done for you!

You will implement Connect4